

1. ¿Qué significa que los arreglos de NumPy sean más eficientes que las listas estándar de Python?

Los *arrays* (arreglos) de NumPy son más eficientes que las listas de Python por varios motivos:

1. Almacenamiento contiguo en memoria

- En un *ndarray* de NumPy, los elementos se guardan en ubicaciones contiguas de memoria (de manera similar a un arreglo en C), lo que facilita el acceso secuencial y reduce la sobrecarga durante las operaciones de lectura y escritura.
- En contraste, las listas de Python son estructuras más generales que pueden almacenar objetos de distinto tipo y en ubicaciones de memoria no necesariamente contiguas.

2. Tipado homogéneo

- Los arreglos de NumPy suelen tener un tipo de dato homogéneo (por ejemplo, `float64`, `int32`, etc.). Esto simplifica la administración interna de datos y permite realizar cálculos vectorizados de manera más eficiente.
- Las listas de Python pueden contener elementos de diferentes tipos, lo cual hace que ciertas operaciones no puedan optimizarse de la misma forma.

3. Operaciones vectorizadas y uso de rutinas internas en C/Fortran

- NumPy implementa muchas de sus operaciones matemáticas usando rutinas de bajo nivel optimizadas (en C o Fortran). Esto permite realizar operaciones (suma, multiplicación, funciones trigonométricas, etc.) de manera masiva sobre grandes cantidades de datos con mucho mejor rendimiento.
- En las listas de Python, para lograr el mismo comportamiento, generalmente se debe recurrir a bucles explícitos en Python, que suelen ser bastante más lentos.

En resumen, la contigüidad en memoria, el tipado homogéneo y la optimización de rutinas matemáticas en bajo nivel son las principales razones por las que los arreglos de NumPy son más rápidos y eficientes.

2. ¿Qué significa `axis=0` y `axis=1` en NumPy? ¿Cómo afectan estas opciones a las operaciones como la suma o el promedio?

En NumPy, el parámetro `axis` sirve para indicar a lo largo de cuál dimensión se aplicará la operación.

- Para un arreglo bidimensional (matriz) *A*, con forma (`n_filas`, `n_columnas`), podemos definir:
 - **`axis=0`**: Significa que la operación se realiza a lo largo de las **filas**, reduciendo la dimensión de las filas y conservando las columnas.
 - Por ejemplo, `A.sum(axis=0)` sumará los valores de cada columna, devolviendo un resultado de tamaño `n_columnas`.

- De forma análoga, `A.mean(axis=0)` calculará el promedio de cada columna.
- **axis=1:** Significa que la operación se realiza a lo largo de las **columnas**, reduciendo la dimensión de las columnas y conservando las filas.
 - Por ejemplo, `A.sum(axis=1)` sumará los valores de cada fila, devolviendo un resultado de tamaño `n_filas`.
 - `A.mean(axis=1)` calculará el promedio de cada fila.

En arreglos con mayor número de dimensiones (3D, 4D, etc.), el valor de `axis` indica de igual forma el índice de la dimensión sobre la que se realiza la operación. Por ejemplo, en un arreglo 3D con forma `(d1, d2, d3)`, `axis=0` se refiere a la dimensión `d1`, `axis=1` a `d2`, y así sucesivamente.

3. ¿Cómo se pueden seleccionar todas las filas o columnas de una matriz en NumPy?

Para un arreglo bidimensional `A` de forma `(n_filas, n_columnas)`, la indexación de NumPy permite:

1. **Seleccionar todas las filas de una columna específica**
 - Para obtener la columna `j` (contando desde 0), se utiliza `A[:, j]`.
 - El símbolo `:` (dos puntos) significa “todas las filas” en esa dimensión.
2. **Seleccionar todas las columnas de una fila específica**
 - Para obtener la fila `i`, se utiliza `A[i, :]`.
 - Aquí `:` significa “todas las columnas”.
3. **Seleccionar todas las filas y todas las columnas**
 - Para obtener la **matriz completa**, simplemente `A[:, :]` (aunque la mayoría de las veces `A` sin más también funciona).
 - Si quisiéramos, por ejemplo, las filas desde `i1` hasta `i2-1` y todas sus columnas, podríamos hacer `A[i1:i2, :]`.
 - Para columnas desde `j1` hasta `j2-1`, haríamos `A[:, j1:j2]`.

Además, NumPy permite indexación avanzada (boolean indexing y fancy indexing) para seleccionar subconjuntos más complejos basados en condiciones o listas de índices.

4. ¿Qué son las funciones universales (ufuncs) en NumPy y cómo se usan?

Las **ufuncs** (*universal functions*) son funciones vectorizadas que operan elemento a elemento sobre los ndarrays. Algunas características clave:

1. **Operan sobre arrays enteros, flotantes, etc.**
 - Por ejemplo, `np.add`, `np.subtract`, `np.multiply`, `np.sin`, `np.exp`, `np.log`, entre muchas otras, son todas ufuncs.

2. Vectorización implícita

- Una ufunc aplica la misma operación a cada elemento del array sin necesidad de escribir bucles explícitos en Python. Esto reduce el tiempo de ejecución y el riesgo de errores típicos de los bucles manuales.

3. Soporte para *broadcasting*

- Muchas de estas funciones soportan el llamado “broadcasting” de NumPy, lo que les permite operar con arrays de distintas dimensiones siempre y cuando las dimensiones sean compatibles según las reglas de broadcasting.

4. Ejemplos de uso

- `np.sin(A)`: Calcula el seno de cada elemento del array A.
- `np.log(A)`: Calcula el logaritmo natural de cada elemento de A.
- `np.add(A, B)`: Suma elemento a elemento dos arrays A y B (si son compatibles en sus formas).

Las ufuncs generalmente soportan parámetros opcionales como `out` para especificar un arreglo de salida donde se guardarán los resultados (ahorrando memoria), y `where` para filtrar sobre qué elementos se aplica la operación, entre otros.